

SSA Register Allocator — Coloring Phase Design

1. Motivation

AMDGPU kernels are register-pressure sensitive: occupancy (number of concurrent wavefronts) is inversely proportional to VGPR/SGPR usage. The existing greedy register allocator destroys SSA form before allocation, losing structural information that guarantees polynomial-time optimal coloring.

An SSA-based allocator can exploit the following theoretical result:

Theorem (Hack, Grund & Goos, CC'06): The interference graph of a program in strict SSA form is *chordal*. Every chordal graph admits a *perfect elimination ordering* (PEO), and greedy coloring on a PEO uses the minimum number of colors (= the *chromatic number* = the *clique number*).

This means register allocation on SSA programs reduces to: 1. Compute a PEO. 2. Color greedily in reverse PEO.

No backtracking, no spilling decisions during allocation, no NP-hard graph coloring — the result is optimal in the number of simultaneously live physical registers.

Reference

Sebastian Hack, Daniel Grund, Gerhard Goos. *Register Allocation for Programs in SSA-Form*. Compiler Construction (CC), 2006. [\[\[06-Research/Papers/ssara.pdf|PDF\]\]](#)

2. Theoretical Foundation

2.1 SSA Interference Graphs Are Chordal

See [\[\[03-Concepts/Chordal_Graphs|Chordal Graphs\]\]](#) for the definition and consequences.

The key result for register allocation: in strict SSA form every virtual register has exactly one definition, so its live range is a connected subtree of the dominance tree. The intersection graph of subtrees of a tree is always chordal — this is a theorem, not an approximation.

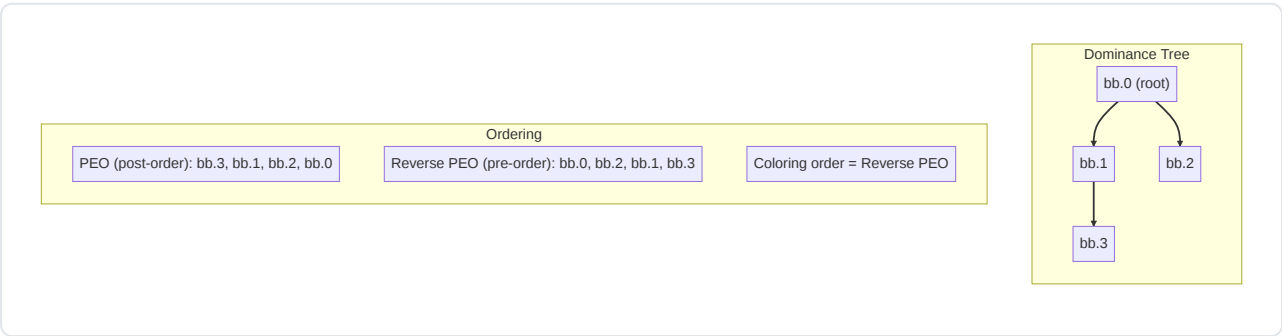
2.2 PEO from the Dominance Tree

See [\[\[03-Concepts/Perfect_Elimination_Order_\(PEO\)|Perfect Elimination Order \(PEO\)\]\]](#) for the general definition.

For SSA interference graphs, a PEO is obtained directly from the dominance tree without running MCS or LexBFS:

- **PEO**: post-order traversal of the dominance tree (children before parent).
- **Reverse PEO** (coloring order): pre-order traversal (parent before children).

The implementation uses `depth_first(MDT->getRootNode())` which produces dominance pre-order — exactly the reverse PEO needed for greedy coloring.



2.3 Greedy Coloring Guarantee

When vertices are processed in reverse PEO, each vertex's already-colored neighbors form a clique (by the PEO property). Greedy assignment of the smallest available color therefore uses at most $\omega(G)$ colors, where $\omega(G)$ is the maximum clique size — which equals the chromatic number for chordal graphs.

3. AMDGPU-Specific Challenge: Mixed Register Widths

AMDGPU programs use registers of multiple widths simultaneously:

Register Class	Width	Example Physical Register
VGPR_32	32-bit	VGPR0
VReg_64	64-bit	VGPR0_VGPR1
VReg_128	128-bit	VGPR0_VGPR1_VGPR2_VGPR3
VReg_256	256-bit	VGPR0_VGPR1_..._VGPR7

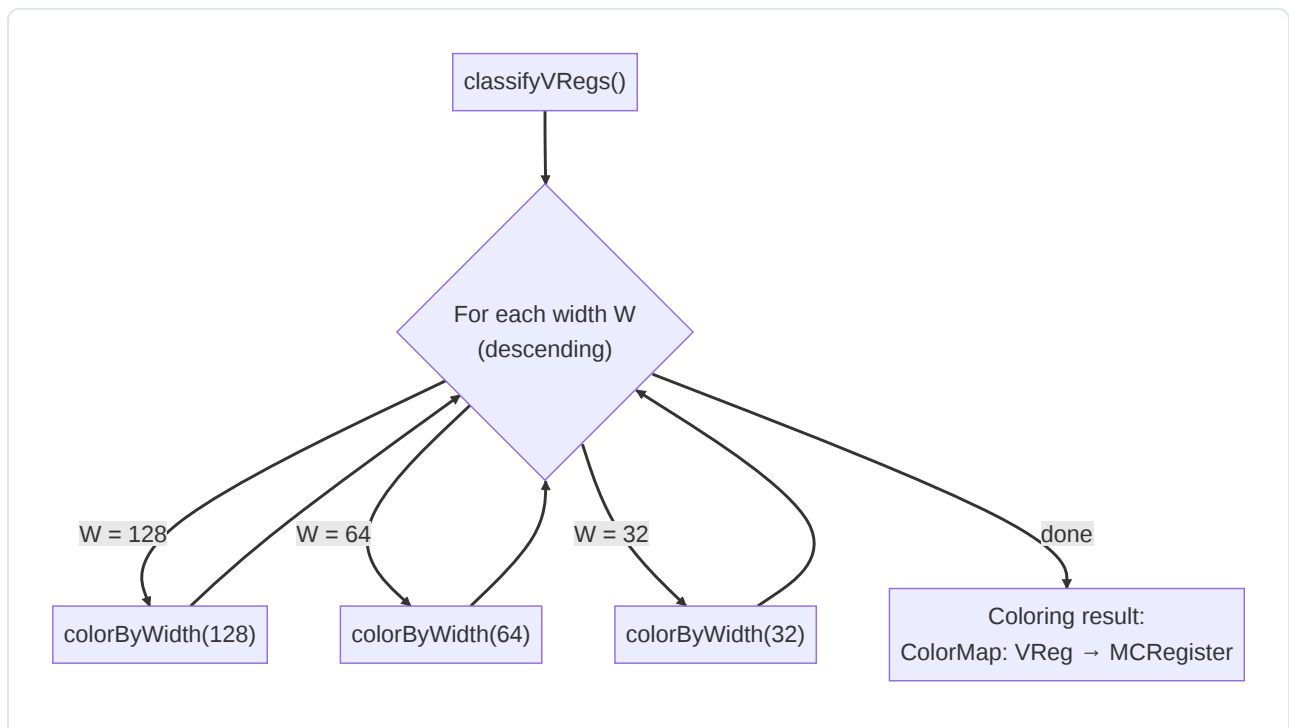
A single 128-bit allocation blocks four 32-bit slots. Naively interleaving widths in a single pass causes fragmentation: narrow allocations may prevent later wide allocations from finding contiguous slots, even

when the total pressure is within limits.

4. Algorithm: Width-Descending Multi-Pass Coloring

4.1 Overview

The coloring runs one complete dominance-tree walk **per distinct register width**, processing widths from widest to narrowest.



4.2 Why Width-Descending?

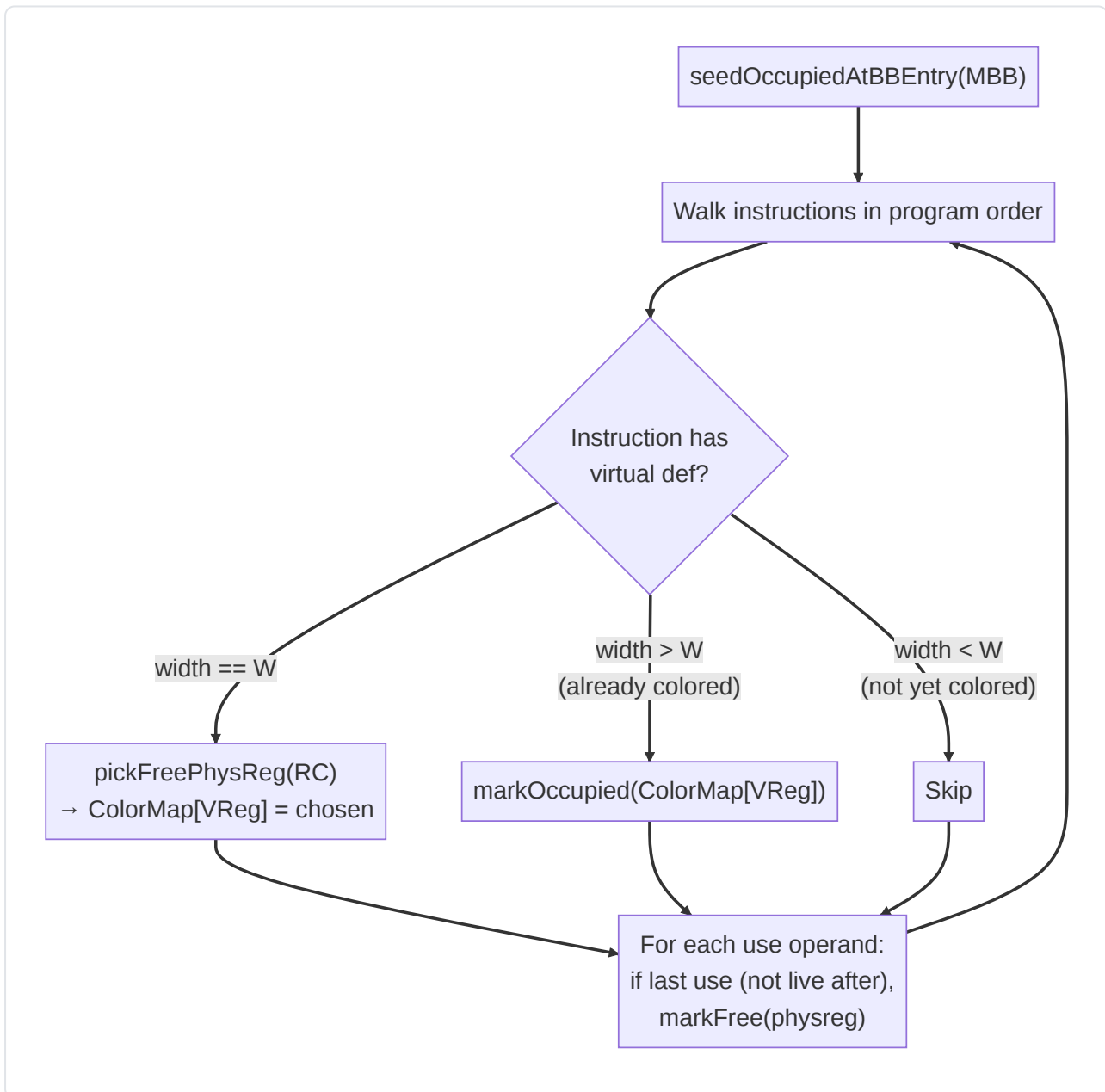
Problem: If 32-bit vregs are colored first, they may scatter across the register file. A later 128-bit vreg needing 4 contiguous slots may find no available aligned tuple, even though 4 individual slots are free.

Solution: Coloring widest first guarantees wide tuples see an unfragmented register file. Narrow vregs colored later fill gaps between wide allocations.

Key insight: During narrow-width passes, freed wide-register slots are immediately available for reuse. When a 128-bit vreg dies mid-block, its 4 constituent reg units are released, and subsequent 32-bit defs in the same block can reuse those slots. This makes a separate post-coloring "splitter" pass unnecessary — the coloring itself achieves optimal packing within each width pass.

4.3 Per-Block State: `colorByWidth(width)`

For each BB in dominance pre-order:



4.4 seedOccupiedAtBBEntry

At each block entry, the occupied-register state is reconstructed from scratch:

1. Clear `OccupiedRegUnits` bitvector.
2. For every `(VReg, PhysReg)` in `ColorMap` : if `LIS->getInterval(VReg).liveAt(BBStart)` , mark `PhysReg`'s reg units as occupied.
3. Mark physical live-ins (e.g., `$exec`) as occupied.

This reconstruction is correct because SSA guarantees each vreg has exactly one live range (one segment in `LiveIntervals`). Checking `liveAt(BBStart)` is sufficient.

4.5 Handling Wider Defs in Narrower Passes

When processing width W and encountering a def of width $W' > W$: - The vreg was already colored in an earlier pass. - Its reg units must be marked as occupied **at its def point**, not just at block entry. This is because the vreg may be born mid-block (not live-in), so `seedOccupiedAtBBEntry` wouldn't catch it.

4.6 Tied Operands

For instructions with tied def-use pairs (e.g., `$dst = V_MAC_F32 $src0, $src1, $dst(tied)`), the def inherits the use's physical register:

```
if (MI.isRegTiedToUseOperand(DefOpIdx, &UseOpIdx))
    Chosen = ColorMap.lookup(MI.getOperand(UseOpIdx).getReg());
```

This preserves the hardware constraint that tied operands share the same physical register.

5. Data Structures

5.1 `ColorMap: DenseMap<Register, MRegister>`

Maps each virtual register to its assigned physical register. Populated during coloring, consumed by the (future) operand-rewrite phase.

5.2 `OccupiedRegUnits: BitVector`

Sized to `TRI->getNumRegUnits()`. Each bit corresponds to one `MRegUnit`. Used for $O(1)$ interference checking: a physical register is free iff none of its constituent reg units are set.

Why reg units? Reg units are LLVM's canonical way to represent hardware register overlap. A 128-bit register like `VGPR0_VGPR1_VGPR2_VGPR3` decomposes into 4 reg units. Checking/setting at the reg-unit level automatically handles all overlap cases without explicit tuple-intersection logic.

5.3 `ColoringOrder: std::set<unsigned, std::greater<unsigned>>`

Sorted set of distinct register widths (in bits) present in the function, ordered descending. Drives the multi-pass loop.

6. Design Decisions

D1: Width-Descending vs. Single-Pass

Approach	Pros	Cons
Single pass (all widths)	One MDT walk	Fragmentation: narrow defs block wide tuples
Width-descending	No fragmentation, optimal packing	Multiple MDT walks (one per width)

Decision: Width-descending. The extra MDT walks are cheap (linear in IR size). Fragmentation avoidance is critical for AMDGPU where 128/256-bit tuples are common.

D2: Per-Point Liveness via Reg Units (not Explicit Interference Graph)

The classical approach builds an interference graph and then colors it. We skip graph construction entirely:

- Interference is tracked implicitly via `OccupiedRegUnits` updated at each program point.
- The MDT pre-order walk ensures we process blocks in reverse PEO.
- Within a block, we process instructions in program order, maintaining accurate per-point liveness.

This avoids $O(V^2)$ graph construction and achieves $O(V \times U)$ coloring where V = number of vregs and U = average reg units per register.

D3: Splitter Removal

An occupancy-driven LR splitting phase was designed and implemented, then removed after empirical analysis showed it was redundant:

- The width-descending coloring already reuses freed register slots at def points. When a wide vreg dies and a new vreg is born later in the same block, the coloring phase assigns the freed slot immediately.
- The splitter's main value proposition — splitting a long-lived wide vreg to free slots for narrower vregs — is already achieved by the coloring's natural per-point liveness tracking.
- Removed ~330 lines of infrastructure (`ShadowMap`, `planSplit`, `commitPlan`, `gap-filling`, `recoloring`) with no test regressions.

D4: No SGPR/VGPR Separation in Pass Structure

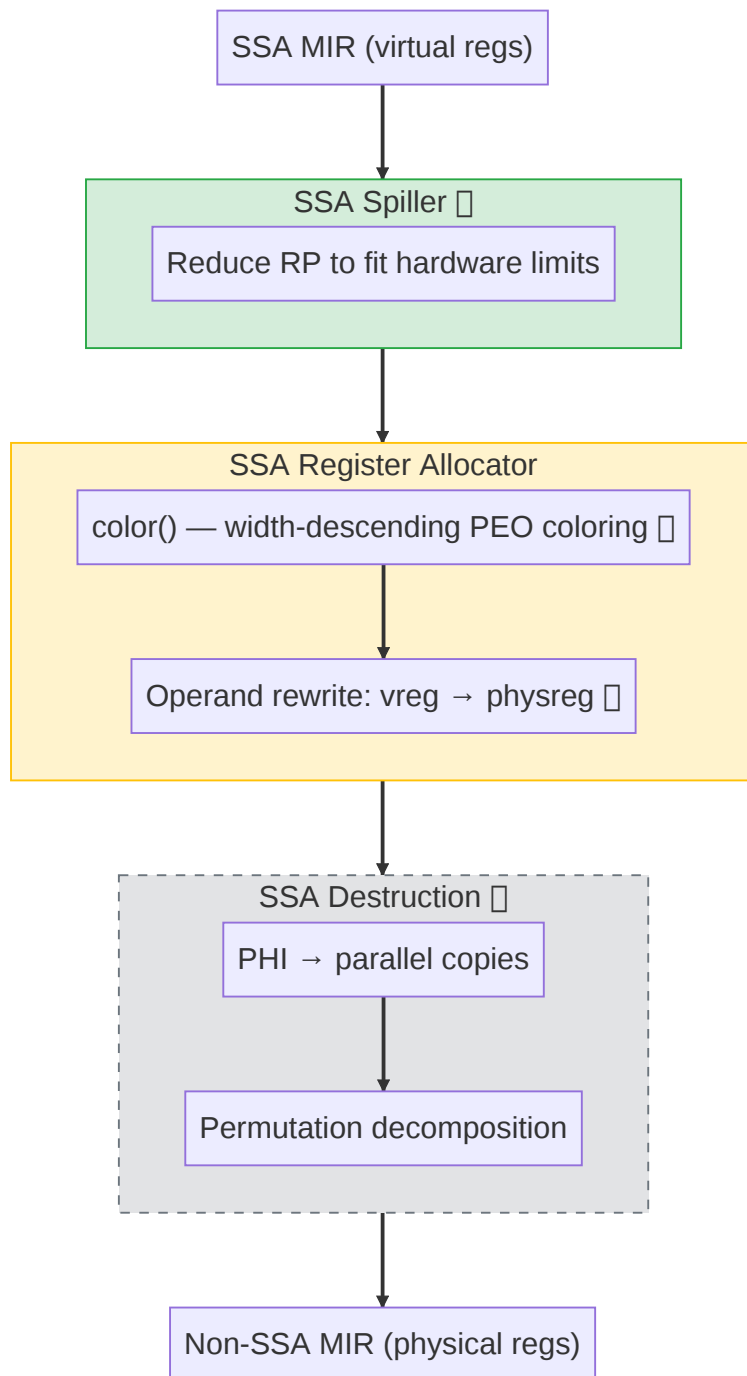
SGPR and VGPR register units occupy disjoint namespaces in LLVM's reg-unit scheme. A VGPR allocation never conflicts with an SGPR allocation at the reg-unit level. Therefore, a single pass handles all register files without explicit class separation.

D5: SSA $\Rightarrow$ Single Segment per VReg

In SSA form, each vreg has exactly one definition. Therefore each vreg's `LiveInterval` consists of a single segment. This simplifies:

- Liveness queries: `liveAt(SlotIndex)` is a single-range check.
- Kill detection: a use is a kill iff the vreg is not live after the use slot.
- No need to iterate subranges for liveness decisions.

7. Pipeline Position



See also: [\[\[03-Concepts/SSA_Destruction|SSA Destruction\]\]](#) for the theory behind PHI lowering as permutation synthesis.

8. Correctness Argument

8.1 Width-Descending Preserves PEO Optimality

Claim: For each width W , the coloring produced by `colorByWidth( $W$ )` is optimal (uses minimum colors) for the sub-interference-graph restricted to vregs of width W , given the constraints imposed by already-colored wider vregs.

Argument: 1. The sub-graph of same-width vregs is still chordal (a subgraph of a chordal graph induced by a vertex subset is chordal). 2. The MDT pre-order provides a valid PEO for this sub-graph. 3. Wider vregs' reg units are marked occupied, acting as additional constraints. These constraints are fixed (not part of the greedy choice), so the greedy coloring on the PEO remains optimal for the remaining degrees of freedom.

8.2 Correctness of Freed-Slot Reuse

When a vreg dies (last use) mid-block, `markFree()` releases its reg units. A subsequent def in the same block may reuse those units. This is correct because: - The dying vreg's live range ends before the new def's live range begins. - They do not interfere. - The PEO ordering ensures all interfering vregs are already colored before the current vertex.

9. Example: Width-Descending Coloring

```
bb.0:
  %A:vreg_128 = IMPLICIT_DEF          ; 128-bit
  %B:vreg_128 = IMPLICIT_DEF          ; 128-bit
  S_NOP 0, implicit %A                ; %A dies
  %C:vgpr_32 = V_MOV_B32 0             ; 32-bit, born after %A dies
  S_BRANCH %bb.1

bb.1:
  S_ENDPGM 0, implicit %B, implicit %C
```

Pass 1 (128-bit): - `%A` → `VGPR0_VGPR1_VGPR2_VGPR3` - `%B` → `VGPR4_VGPR5_VGPR6_VGPR7` - `%A` dies → free units 0–3

Pass 2 (32-bit): - At `%C`'s def: mark `%B` occupied (wider, already colored). `%A` is dead. - `%C` → `VGPR0` (reuses freed slot from `%A`)

Result: 8 VGPRs used (VGPR0–VGPR7), not 9. No splitter needed.

10. Current Status and Future Work

Item	Status
Width-descending PEO coloring	☐ Implemented
Tied operand handling	☐ Implemented
Physical live-in tracking	☐ Implemented
kills-before-defs ordering	☐ Implemented
Physreg defs/kills in colorByWidth	☐ Implemented
Operand rewrite (vreg → physreg)	☐ Implemented
SSA Destruction (PHI lowering)	☐ Implemented (lowerPHIs + resolvePermutation: scratch / V_SWAP_B32 / XOR)
25 LIT tests (coloring + destruction + physreg)	☐ Passing
Pipeline integration (lowerSGPRSpills)	☐ Accounting done; materialization via SLowerSGPRSpills
Pipeline wiring (-amdgpu-ssa-regalloc)	☐ Pending
PHI Coalescing (paper §4.3)	☐ Pending
Loop-filter fallback (validateFinalRP)	☐ Pending (2 XFAIL tests)

11. Source Files

File	Description
<code>llvm/lib/Target/AMDGPU/AMDGPUSSARegisterAllocator.h</code>	Pass class, data structures
<code>llvm/lib/Target/AMDGPU/AMDGPUSSARegisterAllocator.cpp</code>	Coloring implementation (~200 LOC)
<code>llvm/test/CodeGen/AMDGPU/SSARA/*.mir</code>	12 MIR test files